Agentic Digital Twin for Predictive Maintenance

in Manufacturing Using Machine Learning and Generative AI

Md Abu Yusuf; Vu Quynh Dinh
CS 582 Machine Learning
October 19, 2026

Abstract

The innovation involved in the project is the introduction of an agentic Manufacturing Copilot with the aid of a machine learning digital twin. On the AI4I 2020 synthetic dataset, we assessed the performance of Random Forest and XGBoost on a stratified test set containing 2,000 records. The results indicate that the Random Forest model has a precision rate of 91.67%, a recall rate of 80.88%, and an F1 score of 85.94% in identifying machine failure and is used as the active twin model. The Manufacturing Copilot accesses evaluated readings, simulates scenarios, and provides explanations based on the tool's evidence. In 60 evaluation tests performed thrice, GPT-4.1 mini was successful in 145 out of 180 strict execution trials (80.56%). All 30 numerical scenario output results were accurate in comparison with the reference.

1 Introduction

Motivation and business purpose

Unwanted stoppage and maintenance can be a reason for the system which will enable engineers to interpret the machine reading before making any action. The goal of the company is to give priority to the inspection process, perform comparison between hypothetical changes, and present information from the model by using a conversational interface. This prototype is meant to decrease the amount of work necessary for going from reading to discussing maintenance.

Related work

Chen et al. review machine learning in predictive-maintenance digital twins [1]. Luo et al. combine twin models and data for CNC tool-life prediction [2]. Matzka studies explainable models and interfaces [3]. These works motivate prediction and explanation in our project. We use Random Forest [4], XGBoost [5], and LangGraph for copilot orchestration [6].

Project objectives and novelty

Our aims are machine failure classification, evaluation of the virtual state, natural language inspections and what-if reasoning, and the evaluation of predictors' performance and the agent's operation. Our innovation is in the incorporation of an agentic Manufacturing Copilot in the predictive twin. This will choose the right tools for machine state, history, scenario, and previously saved results of the model, explaining the reason behind. Random Forest calculates the scores; the agent manages the process. This is our implementation and its evaluation, without a claim of inventing any new classifier or something unprecedented in the world.

Our scope includes one virtual machine with manual data input and evaluated states. Live data from sensors, life prediction, and physical control are beyond the implementation. Model-based comparisons serve to discuss maintenance; their impact on the system is not established.

2 Dataset and Data Preparation

Dataset and exploratory data analysis

AI4I 2020 contains 10,000 synthetic records: 9,661 normal observations and 339 failures (3.39%), with no missing values [7]. Six inputs comprise product quality (L/M/H), air and process temperature (K), speed (rpm), torque (Nm), and tool wear (min). The binary target indicates machine failure.

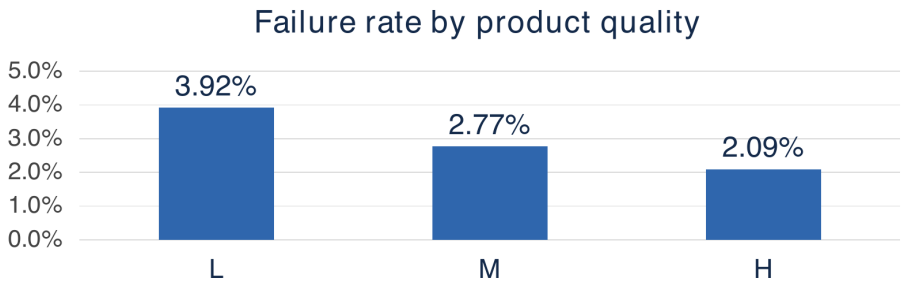


Figure 1 EDA failure rate by product quality in the full dataset [7]

Figure 1 compares within-group failure rates: L 3.92%, M 2.77%, and H 2.09% [7]. L has the highest observed rate; this association does not establish causation. Other EDA uses class counts, boxplots, and torque-speed scatterplots. Failure records show higher median torque and wear, and lower speed. Torque-speed correlation is -0.875; outliers are retained. An always-normal baseline gives 96.61% accuracy and zero failure recall.

Leakage control and one hot encoding

We exclude UDI, Product ID, and failure-mode labels TWF, HDF, PWF, OSF, and RNF to prevent identifier and target leakage. pandas.get_dummies one-hot encodes quality in fixed L/M/H order: L = (1, 0, 0), M = (0, 1, 0), H = (0, 0, 1). All three columns remain when a batch lacks a category; unknown levels are rejected. Training, prediction, and simulation share this encoding, which learns no test-data statistics. Five numeric inputs, six engineered features, and three indicators yield 14 model inputs.

Feature	Definition
Temperature difference	Process temperature minus air temperature
Temperature ratio	Process temperature divided by air temperature
Mechanical power	Torque times angular speed, converted to kW
Normalized torque	Torque divided by speed in thousands of rpm
Wear and torque interaction	Tool wear multiplied by torque
Wear and power interaction	Tool wear multiplied by mechanical power
Product quality indicators	Separate binary indicators for L, M, and H

Table 1 Target-independent engineered features

Power converts rpm to angular speed and watts to kW; feature formulas require no failure label.

Training and test partition

Stratified split with 80/20 ratio (seed 42) yields 8,000 training samples with 271 failure cases and 2,000 test samples with 68 failure cases. Both models are trained on the same data splits with the same features and class weights. No resampling and scaling was done to any dataset. Dataset EDA has been performed before the experiment.

3 Predictor Training and Results

Random Forest model has 300 trees, balanced classes and random seed = 42. XGBoost model has 300 trees, tree depth = 4, learning rate = 0.05, row and column sample = 0.8, and histogram. Its positive class weight is calculated from the ratio of training normal vs failures = 7729/271 = 28.52. The two models have a classification threshold set to 0.5. No hyperparameter search was done in the conducted experiment.

Metric	Random Forest	XGBoost
Accuracy	99.10%	98.50%
Failure precision	91.67%	75.00%
Failure recall	80.88%	83.82%
Failure F1	85.94%	79.17%
ROC AUC	0.9686	0.9821
Average precision	0.8606	0.8902

Table 2 Saved predictive performance on 2,000 test records

Actual class	Random Forest		XGBoost	
	Predicted normal	Predicted failure	Predicted normal	Predicted failure
Normal (1,932)	1,927 (TN)	5 (FP)	1,913 (TN)	19 (FP)
Failure (68)	13 (FN)	55 (TP)	11 (FN)	57 (TP)

Table 3 Confusion matrices on the same 2,000 test records

The rows are observed classes and the columns are predictions; the positive class is failure. TN and TP represent correct predictions of normal and failure, respectively. FP represents false alarm, while FN represents missing failures. Precision refers to the percentage of correctly predicted failures, recall refers to the percentage of failures detected, and F1 balances both.

Why we selected Random Forest

In the current prototype, we have given preference to accurate failure detections and an optimal precision-recall trade-off. The precision rate of failure detection in Random Forest is better (91.67% compared to 75.00%) and so is its F1 score (85.94% compared to 79.17%). Its accuracy rate is also greater (99.10% compared to 98.50%), but this is less important because the number of failures is small. The number of false alarms of Random Forest is 5 compared to 19 of XGBoost, meaning that there are 14 fewer false alarms with 13 more failures detected than 11.

XGBoost algorithm identifies 57 out of 68 cases where there are failures, which is higher than the 55 detected by Random Forest. Recall score is 83.82% for XGBoost and 80.88% for Random Forest, whereas ROC AUC score is 0.9821 for XGBoost and 0.9686 for Random Forest, and finally average precision is 0.8902 for XGBoost and 0.8606 for Random Forest. This selection is based on our prototype preference and not necessarily better in all metrics.

Both classifiers employ a cutoff value of 0.5, which is the same as 50 on the chart showing the risk score value. The Failure category is merely an output from the classifier and does not represent proof of mechanical failure.

4 Digital Twin and Copilot Design

DigitalTwin validates six dashboard inputs and applies the training feature builder. Random Forest calculates a score; the twin stores inputs, classification, indicators, and evaluation time for the copilot.

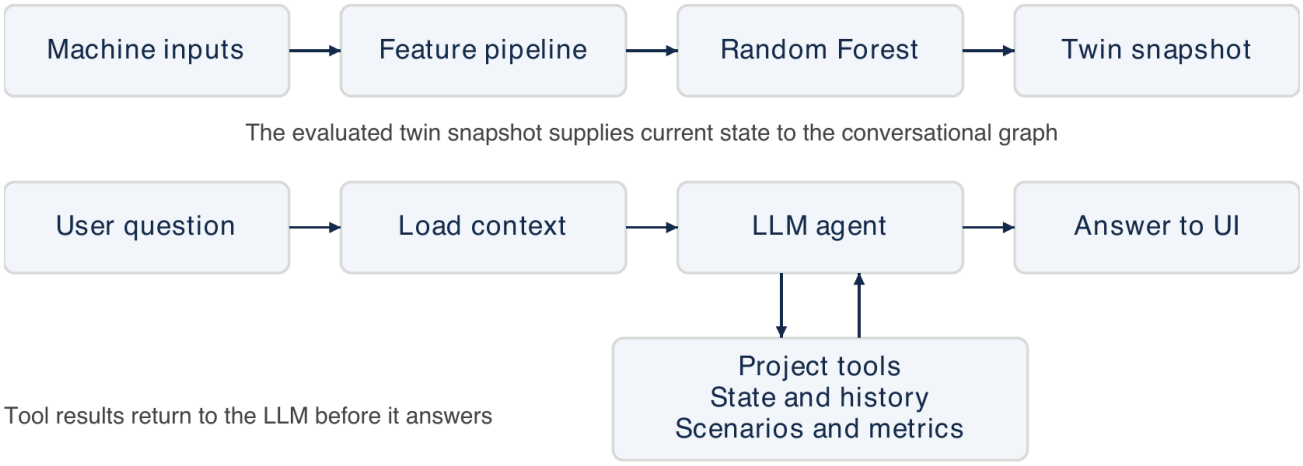


Figure 2 System architecture and conversational execution loop

Figure 2 shows the prediction path and the conversational execution loop. The evaluated twin snapshot supplies current state to the graph, and tool results return to the LLM before it answers.

UpdateState validates and evaluates inputs; PredictFailure returns the prediction; CalculateHealth returns inverse risk. Simulate preserves the baseline; tool replacement changes only wear to zero.

LangGraph nodes and available tools

Three nodes implement the graph: load_context supplies fresh twin context; agent uses GPT-4.1 mini to answer or request tools; tools executes the five functions in Table 4 and returns evidence.

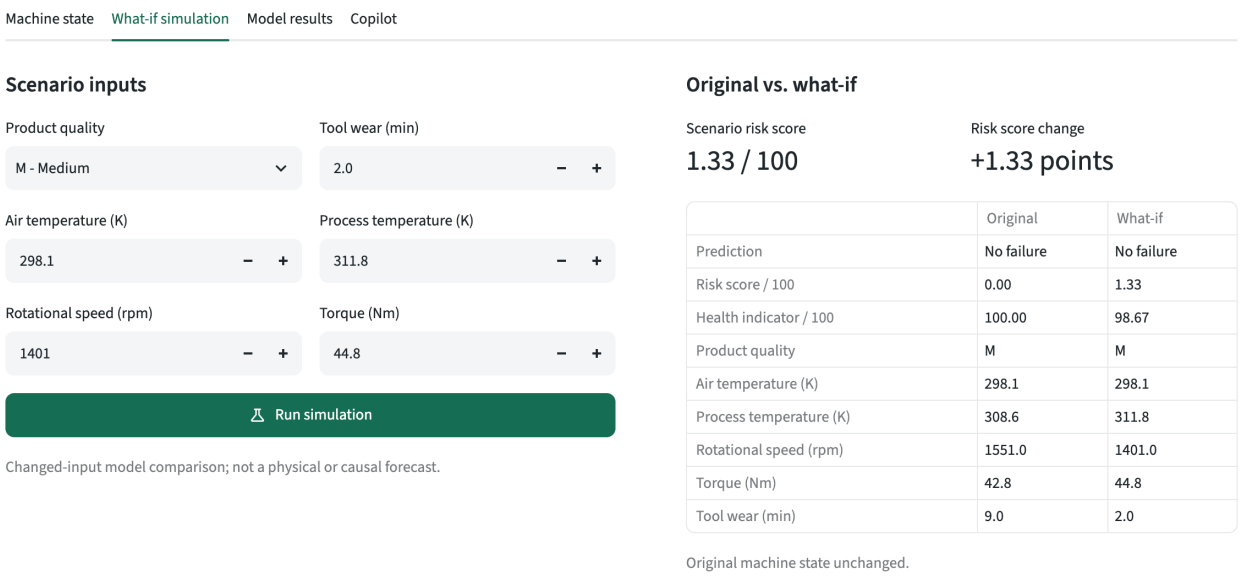
Tool	Evidence or operation
get_current_state	Current evaluated inputs and prediction
get_machine_history	Up to 20 session readings
simulate_scenario	New comparison for requested input changes
get_latest_simulation	Existing comparison for the current baseline
get_model_results	Saved Random Forest or XGBoost test metrics

Table 4 Tools available to the copilot

Fresh context can answer current-risk questions directly. Execution allows six tool rounds and ten completed conversation turns; changing the LLM resets chat. Successful simulations appear in What-if simulation.

Dashboard and Application Output

Figure 3 shows the developed Streamlit application with the Machine state tab selected. This interface displays evaluated model outputs and gives operators access to simulations, saved model results, and the copilot.



	Original	What-if
Prediction	No failure	No failure
Risk score / 100	0.00	1.33
Health indicator / 100	100.00	98.67
Product quality	M	M
Air temperature (K)	298.1	298.1
Process temperature (K)	308.6	311.8
Rotational speed (rpm)	1551.0	1401.0
Torque (Nm)	42.8	44.8
Tool wear (min)	9.0	2.0

Figure 3 Application dashboard with the Machine state tab selected

Dashboard tabs and operator workflow

- **Machine state.** Enter six readings, predict failure, and update the evaluated state and session history.
- **What-if simulation.** Change selected inputs and compare the scenario with a preserved baseline.
- **Model results.** Review saved classifier metrics and confusion matrices for Random Forest and XGBoost.
- **Copilot.** Select the LLM, ask questions, inspect evidence, and download chat. Record two states for history comparisons; the operator decides whether to act.

Recorded question and recommendation

Operator question What if I replace the tool? Compare baseline and scenario risk.

Tool evidence Wear 154 to 0 min; risk 98.33 to 80.33; both classified Failure.

Recorded recommendation excerpt “I recommend inspecting the tool wear condition and proceeding with replacement if wear is confirmed excessive.”

Figure 4 Recorded question and recommendation excerpt from trial 3

Case EVAL-what_if-01 held other inputs fixed. All repeats passed six checks (3.07, 2.93, 2.64 s). This advice does not establish a physical benefit.

5 Copilot Evaluation and Results

Copilot evaluation protocol

A total of 60 tasks were frozen across six different groups, distinct from 12 development tasks. The distinct states that are present in the test partition of ML include failure cases, low score normal cases, and threshold boundary cases. The labels help define the states but are never part of the prompts. With three repetitions, using a new twin and dialogue, there are 180 task trials and 210 dialogues including history warm-ups.

Direct twin calls and saved metadata provide references. Checks require a nonempty answer, fresh context, preserved baseline, required tools and exact arguments, no unrequested successful simulation, and matching scenario outputs. Every turn must pass. Verifiable outcomes and repeats follow the motivation of tau-bench [8]; our tasks and checks are project-specific.

Evaluation results and interpretation

Task category	Passed / trials	Pass rate
Current-state explanation	30 / 30	100.00%
What-if simulation	20 / 30	66.67%
History and follow-ups	28 / 30	93.33%
Model metric retrieval	30 / 30	100.00%
Ambiguous or invalid requests	10 / 30	33.33%
Unsupported requests	27 / 30	90.00%
Overall	145 / 180	80.56%

Table 5 Strict automatic execution checks

No execution errors occurred. Mean response time was 2.14 seconds per turn, excluding initialization; 46 of 60 tasks (76.67%) passed all repeats. Resampling whole tasks with their three trials gives a descriptive 95% bootstrap interval of 70.56%-89.44% (10,000 resamples, seed 42), conditional on this stress-test suite.

All thirty required numerical scenario outputs were consistent with references. The strict flag group consists of ten redundant-baseline-argument trials, two missing history-tool trials, twenty trials that are ambiguous or incorrect, requiring explanation or fix, and three guarantee-request trials that pretended to replace while refusing a guarantee. These checks do not indicate the hallucination rate or answer correctness in general.

6 Setup Limitations and Conclusion

Application setup and reproducibility

Install dependencies in a virtual environment in Python and run the dashboard from the project directory. The XGBoost library needs OpenMP for macOS. Set up an OpenAI API key in the dashboard configuration or via the environment/secrets method described in documentation.

```
python -m pip install -r requirements.txt
python -m streamlit run app.py
python -m src.evaluation.run --split evaluation --trials 3
```

There are four tabs available to give machine states, what-if simulations, model outcomes, and co-pilot chats. Every analysis is saved in a separate folder containing transcripts, verifications, timings, and manifests with respect to the GPT-4.1 mini baseline.

Limitations and future work

The data is artificial, and a random division cannot show performance in future sensor streams or on another machine. The scores are not calibrated, and the health score subtracts risk from 100 instead of independently measuring condition. Artificial input perturbations are used to measure sensitivity without physical or causal certainty. The system has neither a feature attribution function nor Remaining Useful Life estimation. It lacks maintenance manual retrieval and automatic machine control.

The copilot package balances categories for stress representation and is not a workload sample. The run uses a model alias instead of pinning a model version. Independent answer grading, an LLM comparison without tools, and a user study are pending. Future work must force scenario values and units, check impact of prompts/tools on development scenarios, calibrate scores on validation, and get a new task set before claiming progress.

Conclusion

The contribution from the project is an agentic Manufacturing Copilot built on top of the predictive digital twin. What is novel about this project is the conversational access to evidence from the machine and model-driven what-if scenario analysis via tool selection and execution. Random Forest obtained 85.94% failure F1 score, whereas GPT-4.1 mini managed to pass 80.56% of execution tasks strictly. The primary shortcoming of the agents noted was the selection of parameters for unfinished requests.

References

[1] C. Chen, H. Fu, Y. Zheng, F. Tao, and Y. Liu. The advance of digital twin for predictive maintenance: The role and function of machine learning. Journal of Manufacturing Systems 71, pp. 581-594, 2023. https://doi.org/10.1016/j.jmsy.2023.10.010.

[2] W. Luo, T. Hu, Y. Ye, C. Zhang, and Y. Wei. A hybrid predictive maintenance approach for CNC machine tool driven by Digital Twin. Robotics and Computer-Integrated Manufacturing 65, 101974, 2020. https://doi.org/10.1016/j.rcim.2020.101974.

[3] S. Matzka. Explainable Artificial Intelligence for Predictive Maintenance Applications. AI4I, pp. 69-74, 2020. https://doi.org/10.1109/AI4I49448.2020.00023.

[4] L. Breiman. Random Forests. Machine Learning 45, pp. 5-32, 2001. https://doi.org/10.1023/A:1010933404324.

[5] T. Chen and C. Guestrin. XGBoost: A Scalable Tree Boosting System. KDD, 2016. https://doi.org/10.1145/2939672.2939785.

[6] LangChain. LangGraph overview. https://docs.langchain.com/oss/python/langgraph/overview. Accessed October 17, 2026.

[7] UCI Machine Learning Repository. AI4I 2020 Predictive Maintenance Dataset, 2020. https://doi.org/10.24432/C5HS5C.

[8] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains, 2024. https://arxiv.org/abs/2406.12045.